Class: TE **Subject: AISC** **Sem-V**

Experiment No: 4

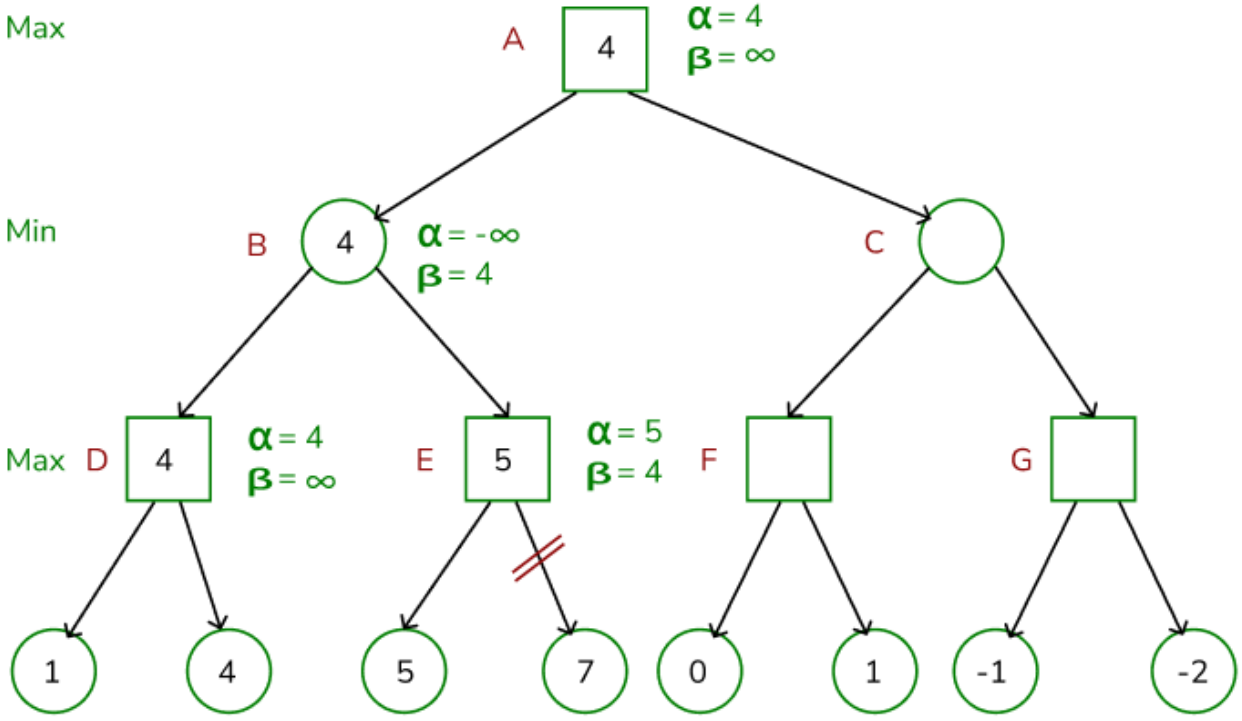
Course Outcome: LO2 **Blooms Level: L3**

Aim: Implement Alpha-Beta Pruning.

Abstract:

Alpha-Beta Pruning is an optimization technique used with the Min-Max algorithm in Artificial Intelligence to improve the efficiency of game-tree search. It reduces the number of nodes evaluated by eliminating branches that cannot influence the final decision, thereby decreasing computation time while producing the same optimal result as the standard Min-Max algorithm. This experiment demonstrates the implementation of Alpha-Beta Pruning using a simple game tree, where alpha and beta values are updated during the search process to prune unnecessary branches. The algorithm is widely used in two-player strategy games such as Chess, Tic-Tac-Toe, Checkers, and Connect Four to make faster and more efficient decisions. This experiment helps students understand adversarial search, recursive tree traversal, and optimization techniques in AI.

Sample Input and Output:



Theory:

Alpha-Beta Pruning is an optimization technique used with the **Min-Max algorithm** in Artificial Intelligence. It reduces the number of nodes evaluated in a game tree by eliminating branches that cannot affect the final decision. The algorithm maintains two values during the search:

- **Alpha (α):** The best value that the MAX player can guarantee.

- **Beta (β):** The best value that the MIN player can guarantee.

During the search:

- At **MAX nodes**, the algorithm updates the alpha value by selecting the maximum score.
- At **MIN nodes**, it updates the beta value by selecting the minimum score.
- Whenever **β ≤ α**, the remaining branches of that node are **pruned (ignored)** because they cannot improve the final result.

Alpha-Beta Pruning produces the **same optimal move as the Min-Max algorithm** but evaluates fewer nodes, making it faster and more efficient. It is widely used in two-player games such as **Chess, Checkers, Tic-Tac-Toe, Connect Four, and Othello**.

Algorithm

1. Start at the root node.
2. Initialize **Alpha = -∞** and **Beta = +∞**.
3. If the current node is a **MAX node**, recursively evaluate its children and update Alpha with the maximum value.
4. If the current node is a **MIN node**, recursively evaluate its children and update Beta with the minimum value.
5. After evaluating each child, check whether **Beta ≤ Alpha**.
6. If **Beta ≤ Alpha**, prune the remaining child nodes of the current node.
7. Continue the process until all required nodes are evaluated.
8. Return the optimal value at the root node.

Code:

```
#include <stdio.h>

#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define MIN(a, b) ((a) < (b) ? (a) : (b))

int alphaBeta(int depth, int nodeIndex, int maximizingPlayer,
              int values[], int alpha, int beta)
{
   // Leaf node
   if (depth == 2)
      return values[nodeIndex];

   if (maximizingPlayer)
   {
```

```
    int best = -999;

    for (int i = 0; i < 2; i++)
    {
        int value = alphaBeta(depth + 1,
                              nodeIndex * 2 + i,
                              0, values, alpha, beta);

        best = MAX(best, value);
        alpha = MAX(alpha, best);

        // Pruning
        if (beta <= alpha)
        {
            printf("Branch pruned at MAX node\n");
            break;
        }
    }

    return best;
}
else
{
    int best = 999;

    for (int i = 0; i < 2; i++)
    {
        int value = alphaBeta(depth + 1,
                              nodeIndex * 2 + i,
                              1, values, alpha, beta);

        best = MIN(best, value);
        beta = MIN(beta, best);

        // Pruning
        if (beta <= alpha)
        {
            printf("Branch pruned at MIN node\n");
            break;
        }
    }

    return best;
}
```

```
}

int main()
{
   // Leaf node values
   int values[] = {3, 5, 2, 9};

   int alpha = -999;
   int beta = 999;

   int result = alphaBeta(0, 0, 1, values, alpha, beta);

   printf("\nBest value = %d\n", result);

   return 0;
}
```

Python Code:

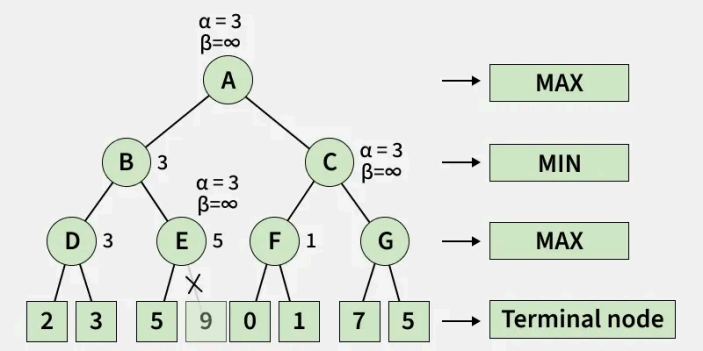
https://colab.research.google.com/drive/17AdeyYRF1GTawnTB_26nmrsZSsdRVGB2?usp=sharing

Conclusion:

In this experiment, Alpha-Beta Pruning was implemented to optimize the Min-Max search algorithm. The algorithm successfully determined the optimal decision while avoiding the evaluation of unnecessary branches. Compared to the standard Min-Max algorithm, Alpha-Beta Pruning significantly reduces computation time without affecting the correctness of the result. Hence, it is an efficient technique for solving adversarial search problems in Artificial Intelligence.

Exercise 1:

Modify the program by changing the leaf node values to:



Run the program and determine:

The optimal value returned by the algorithm.
Which branches are pruned during execution.

Exercise 2:

Modify the program to increase the depth of the game tree from 3 levels to 4 levels (16 leaf nodes). Execute the program with your own leaf values and compare the number of nodes evaluated by:

- Min-Max Algorithm
- Alpha-Beta Pruning

Explain how Alpha-Beta Pruning improves the efficiency of the search.